

Labo 2 - Race Conditions

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)

Februari 2026

1 Context

The goal of this lab exercise was to code in CUDA and find the maximum value in an array using the reduction technique explained in the Powerpoint slides. This assignment further requires of us to think bigger and also explain possible issues that would occur when the array becomes too large to handle for a single block as well as think of a way to implement the reduction methodology for two dimensional arrays.

In order to clearly visualize the advantage of using a GPU and GPU reduction technique we included a visual aid (graph) within this report.

2 Methodology

We solved the reduction algorithm by working with strides, our initial `stride` has a value of half the length of the array, $stride = \frac{len(array)}{2}$. We halve the `stride` by two each iteration and pass it to our CUDA device while $stride > 1$. In essence what our code does is compare the first halve of the array with the second halve and save the maximum values in the first halve. Next iteration we work in quarters, so the first quarter is compared to the second quarter of the array and the maximum values are saved in the first quarter. This halving process continues until our `stride` is less than one, meaning that the first value in our array will contain the maximum value found in our original array. The working of this code can be seen in listing 1.

As you might have noticed our code has a mechanism in place for when our array becomes larger than 2048 (we require additional blocks). With a simple example this can be illustrated, take `stride` 2048 which is halve of `active` and `active` is 4096. Clearly a `stride` of 2048 is to large to handle for a single block in parallel (this can be done sequentially). We calculate the amount of blocks required below, our code casts the floated point value into an int which means the decimal point values are cut off.

$$blocks = \frac{stride + threads - 1}{threads} = \frac{2048 + 1024 - 1}{1024} = 2,999023437 \approx 2$$

If we assume the while loop is executing its first iteration we can assume $active = n$. The original array is 4096 long and requires a maximum of 2048 threads, because there are only 1024 in a block we require 2 blocks. This matches our calculations from above.

Since we wanted to provide an easier comparison between GPU reduction and the simple GPU and CPU algorithms these can be found in listing 2 and 3 respectively. The complete code can be found in section 5.1.

Listing 1: Code snippet, working of the reduction algorithm

```
__global__ void max_reduce_pass(int *a, int n, int stride) {
        int idx = threadIdx.x + blockIdx.x * blockDim.x;
        if (idx < stride && idx + stride < n) {
                a[idx] = max(a[idx], a[idx + stride]);
        }
}


int max_gpu_reduction_global(int *d_a, int n) {
        const int THREADS = 1024;
        int active = n;

        while (active > 1) {
                int stride  = active / 2;
                int blocks  = (stride + THREADS - 1) / THREADS;
                max_reduce_pass<<<blocks, THREADS>>>(d_a, active,
                    stride);
                cudaDeviceSynchronize();  // barrier between
                   passes

                // If active was odd the last element has no
                   partner yet;
                // copy it into slot 0 so it isn't lost.
                // Should be unnecessary since we always use 2^p
                if (active % 2 != 0) {
                        // reuse a single-thread kernel call for
                           the straggler
                        max_reduce_pass<<<1, 1>>>(d_a, active,
                           stride);
                        cudaDeviceSynchronize();
                }
                active = stride;
        }

        int result;
        cudaMemcpy(&result, d_a, sizeof(int),
           cudaMemcpyDeviceToHost);
        return result;
}
```

Listing 2: Code snippet, working of the atomicMax GPU implementation

```
__global__ void max_gpu(int *a, int *r, int n) {
        int idx = threadIdx.x + blockIdx.x * blockDim.x;
        if (idx < n) {
                atomicMax(r, a[idx]);
        }
}
```

Listing 3: Code snippet, working of CPU implentation

```
void max_cpu(int *a, int *r, int n) {
        *r = INT_MIN;
        for (int i = 0; i < n; i++) {
                if (a[i] > *r)
                *r = a[i];
        }
}
```

2.1 Is the result correct?

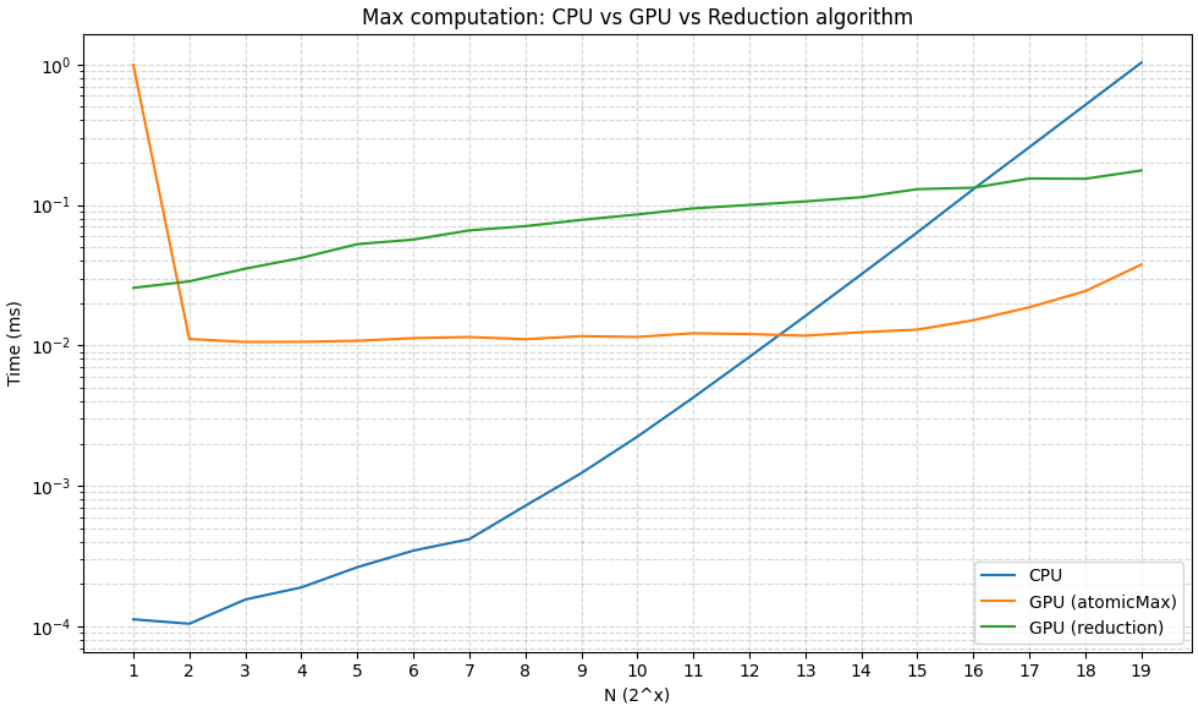
Our code output for all three coding solutions (cpu, gpu and gpu reduction) shows the same maximum values for every iteration we tested. Our tests included all array sizes N where N^p with $0 < p < 20$ (p discreet integer value).

2.2 How does time increase as a function of input size?

If we take a closer look at figure 1 we can clearly see that the CPU is way faster in finding the maximum value in an array when the array is relatively small.

Strangely the GPU (atomicMax) takes a long time to compute the maximum value for an array of size 2^1 which is merely an array of length 2. The most obvious explanation could be a bug in our code or something's going wrong during our duration measurement (likely creating an outlier).

It should be noted that we took an average of 5 measurements of each array size for each of the displayed techniques in order to provide an accurate representation.



Figuur 1: Graph shows a plot based on the increased size of the array

2.3 What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?

As explained before, when the input array is larger than 2048 we would need more than 1024 threads within our single block, this is impossible so we need to split the work over multiple blocks.

Parsing multiple blocks to our method will not guarantee correct working of our program since we cannot synchronize the threads across multiple blocks, we can only synchronize threads within a block.

This brings us to the downside of not using shared memory, since we need to synchronize the CUDA device for each stride, this happens in the CPU and uses the global memory instead of the faster shared memory on the device itself.

As mentioned in the Powerpoint we'll get introduced to shared memory in lab 4, so we decided not to try this in the lab yet.

2.4 Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

We can utilize a grid-stride loop, where the `stride` is equal to the total number of threads in the grid. This allows each thread to perform multiple comparisons sequentially by using the `stride` as an offset. For example, thread 1 would process the element at index 1, then index 1 + stride, then 1 + (2 x stride), and so on. This however would result in a slower execution than just having enough threads but is still a viable solution to ensure all elements get processed.

It should be noted the `stride` we mention here does not have the same meaning as it does in previous texts.

2.5 How would you tackle a 2D array?

We could implement a two dimensional block, a well thought through process for finding the max should be considered in this scenario, we could reuse the idea of finding the max using idx and idx + stride where the stride halves every iterative procedure, and starts as half the value of the maximum dimension of a block.

An alternative approach would be to first convert the two dimensional array into a one dimensional array and then apply the same reduction technique as we would for a one dimensional array.

3 Observations/results

While working on this lab we learned a lot more of how CUDA works as well as experienced difficulties when trying to synchronize multiple blocks (which took us some time to figure out was impossible on the device itself). We also forgot to synchronize threads within a block this gave us quite some trouble since often the code would output the correct result but often it wouldn't.

We assumed the GPU would be faster than the CPU for this task, we clearly underestimated the speed of the CPU for sequential tasks compared to the speed of the GPU.

4 Conclusion

The whole purpose of this lab was to avoid race conditions, still we encountered some issues when trying to synchronize threads across multiple blocks which we found out through asking our professor if this was even possible, this was in fact not.

Since we tried to minimize the use of GPT-models we had to really understand the working of our program which learned us a lot about how CUDA handles threads and blocks as well as how to best work for parallelizing code. In the end we did have to rely on GPT-models after hours of searching for bugs.

5 Appendix

5.1 Code

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
#include <cstdlib>
#include <chrono>
#include <time.h>
#include <fstream>

// ------------------------------------------------------------
// Kernel 1: naive atomicMax across all threads
// ------------------------------------------------------------
__global__ void max_gpu(int *a, int *r, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(r, a[idx]);
    }
}

// ------------------------------------------------------------
// Kernel 2: One pass of the reduction: each thread merges
//           element i with element i+stride, writing the
//           result back in-place.
// ------------------------------------------------------------
__global__ void max_reduce_pass(int *a, int n, int stride) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < stride && idx + stride < n) {
        a[idx] = max(a[idx], a[idx + stride]);
    }
}

// ------------------------------------------------------------
// CPU reference implementation
// ------------------------------------------------------------
void max_cpu(int *a, int *r, int n) {
```

```
35      *r = INT_MIN;
36      for (int i = 0; i < n; i++) {
37          if (a[i] > *r)
38              *r = a[i];
39      }
40  }
41
42  // ----------------------------------------------------------
43  float measure_cpu(int p) {
44      const int N = (int)pow(2, p);
45      int* a = new int[N];
46      int r;
47
48      srand((unsigned)time(NULL));
49      for (int i = 0; i < N; i++)
50          a[i] = std::rand() % 10000000;
51
52      const auto start = std::chrono::steady_clock::now();
53      max_cpu(a, &r, N);
54      const auto end   = std::chrono::steady_clock::now();
55      std::chrono::duration<double> elapsed{end - start};
56      float ms = (float)(elapsed.count() * 1000.0);
57
58      std::cout << "Max␣CPU:␣" << r << std::endl;
59      std::cout << "Elapsed␣time:␣" << ms << "␣ms" << std::endl;
60
61      delete[] a;
62      return ms;
63  }
64
65  // ----------------------------------------------------------
66  float measure_gpu(int p) {
67      const int N = (int)pow(2, p);
68      int* a = new int[N];
69      int r = 0;
70      int *d_a, *d_r;
71
72      srand((unsigned)time(NULL));
73      for (int i = 0; i < N; i++)
74          a[i] = std::rand() % 10000000;
75
76      cudaMalloc((void**)&d_a, N * sizeof(int));
77      cudaMalloc((void**)&d_r, sizeof(int));
```

```
78
79      cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
80
81      // INT_MIN initialized as d_r, is used in max function in
            max_gpu
82      int init = INT_MIN;
83      cudaMemcpy(d_r, &init, sizeof(int), cudaMemcpyHostToDevice);
84
85      cudaEvent_t start_cuda, stop_cuda;
86      cudaEventCreate(&start_cuda);
87      cudaEventCreate(&stop_cuda);
88      cudaEventRecord(start_cuda);
89
90      if (p < 11) {
91          max_gpu<<<1, N>>>(d_a, d_r, N);
92      } else {
93          max_gpu<<<(int)pow(2, p - 10), 1024>>>(d_a, d_r, N);
94      }
95
96      cudaDeviceSynchronize();
97      cudaEventRecord(stop_cuda);
98      cudaEventSynchronize(stop_cuda);
99
100     float ms = 0;
101     cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
102
103     // Cpy result only after kernel has finished
104     cudaMemcpy(&r, d_r, sizeof(int), cudaMemcpyDeviceToHost);
105
106     std::cout << "Max␣GPU:␣" << r << std::endl;
107     std::cout << "Elapsed␣time:␣" << ms << "␣ms" << std::endl;
108
109     delete[] a;
110     cudaFree(d_a);
111     cudaFree(d_r);
112     cudaEventDestroy(start_cuda);
113     cudaEventDestroy(stop_cuda);
114
115     return ms;
116 }
117
118 int max_gpu_reduction_global(int *d_a, int n) {
119     const int THREADS = 1024;
```

```
120     int active = n;  // number of live elements this pass
121
122     while (active > 1) {
123         int stride  = active / 2;
124         int blocks  = (stride + THREADS - 1) / THREADS;
125         max_reduce_pass<<<blocks, THREADS>>>(d_a, active, stride)
                ;
126         cudaDeviceSynchronize();  // barrier between passes
127
128         // If active was odd the last element has no partner yet;
129         // copy it into slot 0 so it isn't lost.
130         // Should be unnecessary since we always use 2^p
131         if (active % 2 != 0) {
132             // reuse a single-thread kernel call for the
                    straggler
133             max_reduce_pass<<<1, 1>>>(d_a, active, stride);
134             cudaDeviceSynchronize();
135         }
136         active = stride;
137     }
138
139     int result;
140     cudaMemcpy(&result, d_a, sizeof(int), cudaMemcpyDeviceToHost)
            ;
141     return result;
142 }
143
144
145 // ------------------------------------------------------------
146 float measure_gpu_reduction(int p) {
147     const int N = 1 << p;   // 2^20 = ~1M elements
148
149     int *a = new int[N];
150     srand((unsigned)time(NULL));
151     for (int i = 0; i < N; i++)
152         a[i] = std::rand() % 10000000;
153
154     // Copy to device, reduction is done in-place so we
155     int *d_a;
156     cudaMalloc(&d_a, N * sizeof(int));
157     cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
158
159     // Time the reduction
```

```
160     cudaEvent_t start, stop;
161     cudaEventCreate(&start);
162     cudaEventCreate(&stop);
163     cudaEventRecord(start);
164 
165     int result = max_gpu_reduction_global(d_a, N);
166 
167     cudaEventRecord(stop);
168     cudaEventSynchronize(stop);
169 
170     float ms = 0;
171     cudaEventElapsedTime(&ms, start, stop);
172 
173     // CPU reference, EXTRA
174     int cpu_max = INT_MIN;
175     for (int i = 0; i < N; i++)
176         if (a[i] > cpu_max) cpu_max = a[i];
177 
178     std::cout << "GPU␣max␣(global␣mem␣reduction):␣" << result  <<
        std::endl;
179     std::cout << "CPU␣max␣(reference):␣␣␣␣␣␣␣␣␣␣␣␣" << cpu_max <<
        std::endl;
180     std::cout << "Match:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣" << (result ==
        cpu_max ? "YES" : "NO") << std::endl;
181     std::cout << "Elapsed␣time:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣" << ms << "␣ms
       " << std::endl;
182 
183     delete[] a;
184     cudaFree(d_a);
185     cudaEventDestroy(start);
186     cudaEventDestroy(stop);
187 
188     return ms;
189 }
190 
191 // ------------------------------------------------------------
192 void write_to_csv(std::ofstream &csv_file, float r_cpu, float
    r_gpu, float r_gpu_red) {
193     csv_file << r_cpu << "," << r_gpu << "," << r_gpu_red << "\n"
        ;
194 }
195 
196 void run_measurements(std::ofstream &csv_file) {
```

```
197     for (int i = 1; i < 20; i++) {
198         std::cout << "\n===␣POWER␣OF:␣" << i << "␣(N=" << (int)
               pow(2,i) << ")␣===" << std::endl;
199 
200         int count = 5;
201         float avg_t_cpu = 0;
202         float avg_t_gpu = 0;
203         float avg_t_gpu_red = 0;
204 
205         for (int j = 0; j < count; j++){
206             avg_t_cpu += measure_cpu(i);
207             avg_t_gpu += measure_gpu(i);
208             avg_t_gpu_red += measure_gpu_reduction(i);
209         }
210 
211         write_to_csv(csv_file, avg_t_cpu/count , avg_t_gpu/count,
                avg_t_gpu_red/count);
212     }
213 }
214 
215 int main() {
216     std::ofstream csv_file("data.csv");
217     csv_file << "cpu,gpu,gpu_red\n";
218 
219     run_measurements(csv_file);
220 
221     csv_file.close();
222     return 0;
223 }
```